

Giorno 13 — CRUD REST completo, Spring Data REST e OpenAPI

Esercizio 1 — API REST completa con Spring Data JPA

Completa la tua API REST migrando dal DAO manuale a Spring Data JPA, sfruttando i metodi built-in del repository e le query derivate dal nome del metodo.

Cosa fare:

- Migrazione: sostituisci il tuo DAO con un'interfaccia `StudenteRepository` che estende `JpaRepository<Studente, Integer>`. Verifica che tutte le funzionalità (`findAll`, `findById`, `save`, `deleteById`) continuino a funzionare.
- Aggiungi al repository questi metodi con query derivate: `findByCognome(String cognome)`, `findByCorsoLaurea(String corso)`, `findByNomeContaining(String parte)`.
- Esponi questi nuovi metodi nel controller aggiungendo endpoint: `GET /api/studenti/cognome/{cognome}` e `GET /api/studenti/corso/{corso}`.
- Implementa `PATCH` su `/api/studenti/{id}`: deve aggiornare SOLO i campi presenti nel body JSON (partial update), lasciando inalterati quelli non specificati. Usa `@RequestBody Map<String, Object>` per ricevere solo i campi da aggiornare.
- Aggiungi la paginazione: l'endpoint `GET /api/studenti` deve accettare parametri `page` e `size` (es. `/api/studenti?page=0&size=5`). Usa `Pageable` di Spring Data. La risposta deve includere informazioni sulla pagina corrente, dimensione e totale.
- Aggiungi l'ordinamento: `/api/studenti?sort=cognome,asc` deve restituire gli studenti ordinati per cognome. Verifica che funzioni combinato con la paginazione.

Consegna: Testa paginazione e ordinamento con Postman. Salva i JSON delle risposte (che mostrano i metadati della paginazione) nel file `README.md` del progetto.

Esercizio 2 — Documentazione con OpenAPI/Swagger

Aggiungi la documentazione automatica all'API usando SpringDoc OpenAPI (Swagger). Una buona API deve essere auto-documentata!

Cosa fare:

- Aggiungi al `pom.xml` la dipendenza `springdoc-openapi-starter-webmvc-ui`. Riavvia e apri `http://localhost:8080/swagger-ui.html`. Cosa vedi?
- Arricchisci la documentazione dell'API usando le annotazioni OpenAPI nel controller. Aggiungi `@Tag(name="Studenti", description="Gestione studenti")` alla classe. Aggiungi `@Operation(summary="Descrizione breve")` a ogni endpoint.
- Aggiungi `@ApiResponse` per documentare i possibili codici di risposta di ogni endpoint. Ad esempio, per `GET /api/studenti/{id}`: 200 OK con lo studente, 404 se non trovato.
- Nella classe `Studente`, aggiungi `@Schema(description="...")` su ogni campo per documentare il significato di ogni proprietà. Usa `@Schema(example="Mario")` per fornire esempi.
- Configura le informazioni generali dell'API: crea un bean `OpenAPI` con titolo, versione, descrizione, e informazioni di contatto (il tuo nome e email).
- Apri Swagger UI e testa direttamente un endpoint `GET` e un `POST` dall'interfaccia. Questo è uno strumento molto usato nei team di sviluppo per testare e comunicare le API!

Consegna: Fai uno screenshot della Swagger UI con la tua API documentata. Poi esporta la specifica OpenAPI in formato JSON aprendo `http://localhost:8080/v3/api-docs` e salvala come `api-spec.json`.